

Performance Analysis of Single-Cycle and Pipelined RISC-V Processor

APROJECT REPORT
SUBMITTED FOR THE
REQUIREMENTS OF THE SUBJECT

(PBCST404)
**COMPUTER ORGANIZATION AND
ARCHITECTURE**
OF
APJ Abdul Kalam Technological University
BY

DEVI KRISHNA KONDIYARA
KARTHIKEYAN(VAS24CS101)
GOWRIBHADRA
KRISHNAKUMAR(VAS24CS114)
AVANY MANOJ (VAS24CS090)
JESWIN C J (VAS24CS122)



(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Department of Computer Science and Engineering

Vidya Academy of Science and Technology

Thalakkottukara, Thrissur - 680501

<http://www.vidyaacademy.ac.in>

MARCH 18 2026

DECLARATION

We, **Devi Krishna Kondiyara Karthikeyan (VAS24CS101), Gowribhadra Krishnakumar (VAS24CS114), Avany Manoj (VAS24CS090) and Jeswin C J(VAS24CS122)**, hereby undertake that the project work entitled Empirical Analysis of Cache Block Size on Spatial Locality and System Hit Rates has been carried out by our group under the valuable guidance of **Ms. Remya Krishna J S, Assistant Professor, Department of Computer Science and Engineering, Vidya Academy of Science and Technology, Thalakkottukara, Thrissur**. This report is formally submitted as a required project component for the Computer Organization and Architecture (PBCST404) course in Computer Science and Engineering, under **APJ Abdul Kalam Technological University**, during the academic year 2025- 2026.

Thrissur

MARCH 2026

Gayatri N C (VAS24CS111)

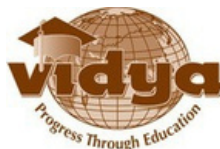
Athulkrishna S (LVAS24CS203)

Asha E R (VAS24CS076)

Devanandhan A B (VAS24CS098)

Department of Computer Science and Engineering Vidya Academy of Science & Technology

Thalakkottukara, Thrissur - 680501



(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Certificate

This certifies that the project report titled "**Performance Analysis of Single-Cycle and Pipelined RISC-V Processor**" represents the genuine work carried out by the group members: Gayatri N C (VAS24CS111), Athulkrishna S (LVAS24CS203), Asha E R (VAS24CS076) and Devanandhan A B (VAS24CS098), of Vidya Academy of Science & Technology, Thalakkottukara, Thrissur - 680 501. This report is formally submitted as a required project component for the Computer Organization and Architecture (PBCST404) course, as part of the B.Tech program (2024-2028 batch) prescribed by **APJ Abdul Kalam Technological University (APJAKTU)**, during the 2025-2026 academic year. The project work has been completed and accepted as meeting the stipulated academic requirements for the subject's project.

Project Guide

Ms. Remya Krishna J S

Assistant Professor, Dept. of CSE

Head of Department

Dr. Sajay K R

Professor, Dept. of CSE

ACKNOWLEDGEMENT

During the course of our mini project, our team of four collaborated closely, and we also received support from various individuals, both directly and indirectly. Without their valuable contributions, completing this work would have been impossible. We would like to take this opportunity to express our sincere gratitude and appreciation for their support. We want to start expressing my thanks to my project guide **Ms. Remya Krishna J S**, Assistant Professor, Dept. of Computer Science and Engineering , because of her valuable advice and guidance towards this work. We received motivation, encouragement and hold up from her during the course of work. We are grateful to express my thanks to all the faculty members of our department for their support. I articulate my gratitude to all my friends for their support and help for this work. We are thankful to **Dr. Sajay K R**, Head of Computer Science and Engineering Engineering Department, and our Principal **Dr. Sunitha C** for their sole co-operation. Last, but not the least we wish to express our gratitude to God Almighty for his abundant blessings without which this effort would not have been successful.

Gayatri N C (VAS24CS111)

Athulkrishna S (LVAS24CS203)

Asha E R (VAS24CS076)

Devanandhan A B (VAS24CS098)

MARCH 2026

Fourth Semester B.Tech (2024 Admissions)

Vidya Academy of Science & Technology

Thrissur - 680 501

ABSTRACT

This project presents a comparative performance analysis of Single-Cycle and 5-Stage Pipelined processor architectures using the Ripes simulator based on the RISC-V architecture. The main objective of this study is to understand how processor design influences execution speed, clock cycles, and overall efficiency. In a Single-Cycle processor, each instruction is executed in one long clock cycle, making the design simple but slower due to increased clock period. In contrast, a Pipelined processor divides instruction execution into five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). This allows multiple instructions to overlap in execution, improving throughput. Using the Ripes simulator, sample RISC-V assembly programs are executed on both processor models. Key performance metrics such as Clock Cycles, Cycles Per Instruction (CPI), and instruction throughput are observed and compared. The study also analyzes the impact of pipeline hazards and stalls on performance. The results demonstrate that while the Single-Cycle processor offers simplicity in design, the Pipelined processor achieves higher efficiency and better performance due to parallel instruction processing. This project provides practical insight into modern processor architecture and helps in understanding the importance of pipelining in improving computational performance.

CONTENTS

CONTENTS	PAGENO:
ACKNOWLEDGEMENT	<i>i</i>
ABSTRACT	<i>ii</i>
LIST OF FIGURES	<i>iv</i>
CHAPTER 1. INTRODUCTION	01
CHAPTER 2. OBJECTIVES	02
CHAPTER 3. SYSTEM OVERVIEW	03
3.1 ARCHITECTURE CONCEPT OVERVIEW	03
3.2 PROBLEM DESCRIPTION	04
CHAPTER 4. SIMULATION ENVIRONMENT	05
4.1 OVERVIEW OF THE SIMULATOR	05
4.2 SYSTEM CONFIGURATION	06
CHAPTER 5. ASSEMBLY SOURCE CODE	08
5.1 ALGORITHM AND FLOWCHART	08
5.2 ASSEMBLY PROGRAM	09
CHAPTER 6. RESULT	11
CHAPTER 7. CONCLUSION	21
CHAPTER 8. REFERENCES	22

LIST OF FIGURES

- 6.1 : Simulation Results for 4-Byte Block Size (Cache Miss)
- 6.2 : Cache graph for 4-Byte Block Size
- 6.3: Simulation Results for 8-Byte Block Size (Cache Hit)
- 6.4: Simulation Results for 8-Byte Block Size
- 6.5: Simulation Results for 16-Byte Block Size (Cache Hit)
- 6.6: Simulation Results for 16-Byte Block Size
- 6.7: Simulation Results for 32-Byte Block Size (Cache Hit)
- 6.8: Cache graph for 32-Byte Block Size
- 6.9: Simulation Results for 64-Byte Block Size (Cache Hit)
- 6.10 Cache graph for 64-Byte Block Size

CHAPTER 1

INTRODUCTION

In modern computer systems, processor performance plays a crucial role in determining the overall efficiency of computing systems. The design of the processor significantly affects execution speed, power consumption, and throughput. Among various processor architectures, Single-Cycle and Pipelined processors are fundamental designs used to understand performance optimization.

A Single-Cycle processor executes each instruction in a single clock cycle. Although this design is simple and easy to implement, it suffers from poor performance due to long clock cycle duration. Each instruction must complete all stages such as fetch, decode, execute, memory access, and write-back in one cycle, resulting in inefficient utilization of hardware resources.

In contrast, a Pipelined processor divides instruction execution into multiple stages. The most common design is a 5-stage pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). This allows multiple instructions to be processed simultaneously, thereby improving throughput and efficiency.

This project focuses on analyzing the performance differences between Single-Cycle and Pipelined RISC-V processors using the Ripes simulator. Key metrics such as clock cycles, CPI (Cycles Per Instruction), and execution time are evaluated. Additionally, pipeline hazards and their effects on performance are studied.

CHAPTER 2

OBJECTIVES

1. To study and understand the working principles of the RISC-V Instruction Set Architecture and its importance in the design and implementation of modern processors.
2. To analyze in detail the architecture and operation of both Single-Cycle and 5-stage Pipelined processors, including how instructions are executed in each stage.
3. To implement and simulate both Single-Cycle and Pipelined processor models using the Ripes Simulator in order to observe their behavior during program execution.
4. To evaluate and compare the performance of both processor architectures based on important metrics such as clock cycles, execution time, and Cycles Per Instruction (CPI).
5. To study how pipelining improves instruction throughput by allowing multiple instructions to overlap during execution, thereby increasing overall efficiency.
6. To analyze the impact of pipeline hazards such as data hazards and control hazards on the performance of the pipelined processor.

CHAPTER 3

SYSTEM OVERVIEW

3.1 ARCHITECTURE CONCEPT OVERVIEW

The architecture concept overview provides the basic understanding required to analyze the performance of different processor designs. In this project, the focus is on the RISC-V Instruction Set Architecture, which is used as the base for simulation and experimentation. Using the Ripes Simulator, the working of both Single-Cycle and Pipelined processors can be observed clearly. The study mainly compares two processor architectures: Single-Cycle processor and 5-stage Pipelined processor. These architectures differ in the way instructions are executed and how efficiently hardware resources are utilized.

The core architectural concepts being modeled include:

1. Single-Cycle Processor

- In a Single-Cycle processor, each instruction is executed in one clock cycle.
- All stages of execution are completed within a single cycle:
 - Instruction Fetch (IF)
 - Instruction Decode (ID)
 - Execute (EX)
 - Memory Access (MEM)
 - Write Back (WB)
- The design is simple and easy to understand.
- The clock cycle duration is very long because it must accommodate all stages.
- This results in lower performance and inefficient hardware utilization.

2. Pipelined Processor

- A Pipelined processor divides instruction execution into multiple stages.
- The standard 5-stage pipeline includes:
 - Instruction Fetch (IF)
 - Instruction Decode (ID)
 - Execute (EX)
 - Memory Access (MEM)
 - Write Back (WB)
- Multiple instructions are executed simultaneously in different stages.
- This overlapping of instructions increases throughput.
- It improves overall processor efficiency compared to Single-Cycle architecture

3. Performance Metrics

- The performance of both processors is evaluated using:
 - Clock Cycles: Total number of cycles required to execute a program
 - Cycles Per Instruction (CPI): Average cycles required per instruction
 - Throughput: Number of instructions executed per unit time
- These metrics help in comparing the efficiency of both architectures.

3.2 PROBLEM DESCRIPTION

The problem description focuses on the challenges involved in improving processor performance through efficient architectural design. While processors are designed to execute instructions quickly, their performance is highly influenced by how instructions are processed and how hardware resources are utilized.

In a Single-Cycle processor, all instructions are executed in a single clock cycle, which results in a long clock period and inefficient performance. Although the design is simple, it does not utilize the processor resources effectively, leading to lower throughput.

To overcome this limitation, pipelining is introduced in processor design. However, while a Pipelined processor improves performance by overlapping instruction execution, it also introduces certain complexities known as pipeline hazards.

The project addresses the following performance challenges:

Low Throughput in Single-Cycle Processor:

The Single-Cycle processor executes only one instruction per clock cycle with a long cycle duration, resulting in reduced execution speed.

Pipeline Hazards:

In a Pipelined processor, hazards occur due to overlapping execution of instructions:

- Data Hazards: When an instruction depends on the result of a previous instruction
- Control Hazards: Caused by branch instructions affecting execution flow
-

Pipeline Stalls:

To resolve hazards, the processor may introduce stalls, which delay instruction execution and reduce performance.

Optimization Goal:

The main goal is to analyze how pipelining improves processor performance compared to the Single-Cycle design and to understand the trade-offs between simplicity and efficiency.

CHAPTER 4

SIMULATION ENVIRONMENT

This chapter describes the experimental setup and tools used to analyze the performance of Single-Cycle and Pipelined RISC-V processors. It explains the simulation platform and system configuration used to carry out the experiments. The analysis is performed using a simulation-based approach, where RISC-V assembly programs are executed on both processor models. The simulator helps in observing internal processor operations such as instruction flow, pipeline stages, and register updates.

4.1 OVERVIEW OF THE SIMULATOR

The primary tool used in this project is the Ripes Simulator. It is a graphical computer architecture simulator designed to demonstrate the working of RISC-V processors in an interactive manner.

Ripes provides a detailed visualization of how instructions are executed inside the processor. It allows users to observe different stages of instruction execution and understand the internal behavior of both Single-Cycle and Pipelined architectures.

Key features of the simulator include:

Visual Processor Interface:

Provides a real-time graphical representation of instruction flow through different stages such as IF, ID, EX, MEM, and WB.

Pipeline Visualization:

Displays how multiple instructions are executed simultaneously in a pipelined processor, making it easier to understand overlapping execution.

Execution Monitoring:

Allows step-by-step execution of assembly programs and helps in tracking register values and memory changes.

Configurable Architecture:

Supports both Single-Cycle and Pipelined processor models for performance comparison.

ISA Support:

Implements the RISC-V Instruction Set Architecture, providing a modern platform for learning processor design concepts.

4.2 SYSTEM CONFIGURATION

This section describes the configuration used in the simulation environment to perform the performance analysis of Single-Cycle and Pipelined processors. It includes details about the processor models, execution setup, and parameters used to evaluate system performance.

Processor Architectures:

- Single-Cycle RISC-V Processor
- 5-Stage Pipelined RISC-V Processor

Pipeline Stages (for Pipelined Processor):

- Instruction Fetch (IF)
- Instruction Decode (ID)
- Execute (EX)

- Memory Access (MEM)
- Write Back (WB)

Simulation Tool:

- Ripes Simulator

Program Executed:

- RISC-V Assembly program used to perform basic operations such as arithmetic and memory access

Performance Metrics Observed:

- Clock Cycles: Total number of cycles required for execution
- Cycles Per Instruction (CPI): Average cycles per instruction
- Instruction Throughput: Number of instructions executed per unit time

Execution Method:

- The same assembly program is executed on both processor models
- Results are observed and compared to analyze performance differences

CHAPTER 5

ASSEMBLY SOURCE CODE

This chapter details the logic and implementation of the data-intensive workload used to test the cache. It describes the sequential array-traversal algorithm designed to stress-test spatial locality and provides the corresponding RISC-V assembly code executed within the Ripes simulator.

5.1 ALGORITHM AND FLOWCHART

The efficiency of a cache is best observed through predictable data access patterns. For this experiment, a sequential array-traversal algorithm is used. This algorithm systematically accesses each element in a large memory block, making it ideal for observing how different cache block sizes capture neighboring data.

Algorithm Logic:

1. Initialize:

- Set a base memory address for the array and a counter for the number of elements.

2. Load:

- Fetch the data from the current memory address into a processor register.

3. Process:

- Perform a simple operation (like an addition or increment) to simulate data processing.

4. Increment:

- Move the memory pointer to the next sequential address (e.g., +4 bytes for an integer).

5. Loop:

- Repeat the process until all elements in the array have been accessed.

6. Terminate:

- End the execution and allow the simulator to calculate the final Cache Hit Ratio.

5.2 ASSEMBLY PROGRAM

The following RISC-V assembly program implements the array-traversal logic described above. It is written to be compatible with the Ripes simulator's processor model.

Code:

```
.data
array: .word 1,2,3,4,5,6,7,8,9,10
       .word 11,12,13,14,15,16,17,18,19,20

.text
.globl main

main:
la t0, array # load address of array
li t1, 20 # number of elements
li t2, 0 # index
```

```
li t3, 0 # sum
```

```
loop:
```

```
lw t4, 0(t0) # load array element
```

```
add t3, t3, t4 # sum += element
```

```
addi t0, t0, 4 # move to next element
```

```
addi t2, t2, 1
```

```
blt t2, t1, loop
```

```
end:
```

```
nop
```

CHAPTER 6

RESULTS

The following sections provide a detailed analysis of the simulation runs conducted using the Ripes RISC-V ISA Simulator. For each configuration, a screenshot of the visual cache management interface is provided to illustrate the real-time interaction between the processor and the memory hierarchy. These visuals include the cache table, which shows data placement and tag matching, and the performance plot, which tracks the cumulative Cache Hit Ratio over the execution cycles. By examining these screenshots sequentially, we can observe the empirical transition from high miss rates at small block sizes to optimized performance as spatial locality is effectively exploited.

CASE-WISE SIMULATION ANALYSIS

Case 1: 4-Byte Block Size (1 Word/Line)

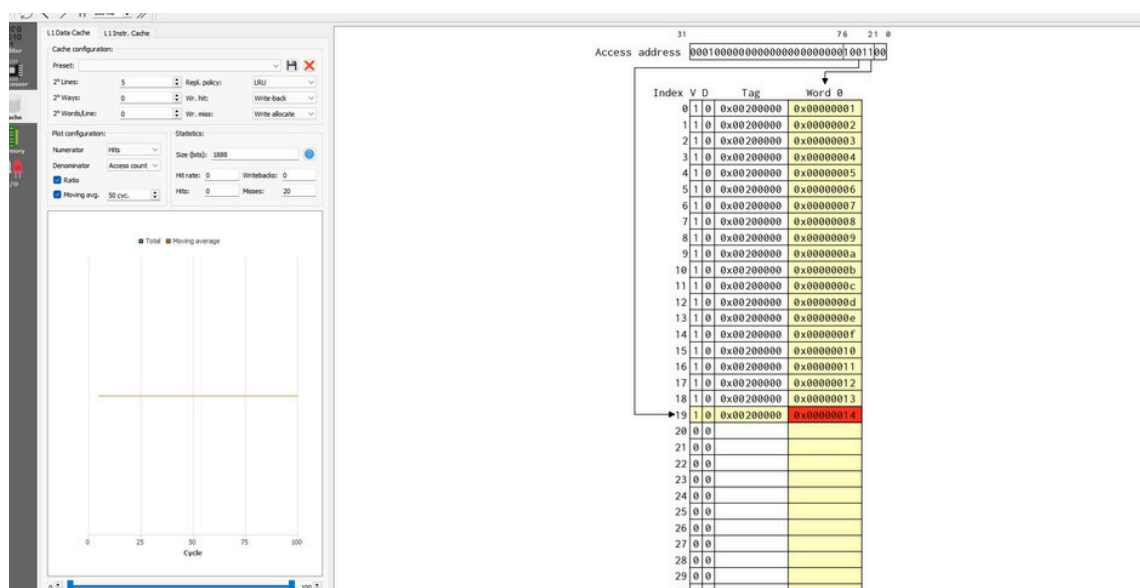


Figure 6.1 : Simulation Results for 4-Byte Block Size (Cache Miss)

- **Configuration:** Block size is set to 4 bytes (2^0 words/line).
- **Results:** Hits: 0, Misses: 20, Hit Rate: 0%.
- **Explanation:** Because the block size matches the word size, each memory access only fetches the exact data requested. There is no "extra" data brought in to satisfy future sequential requests.

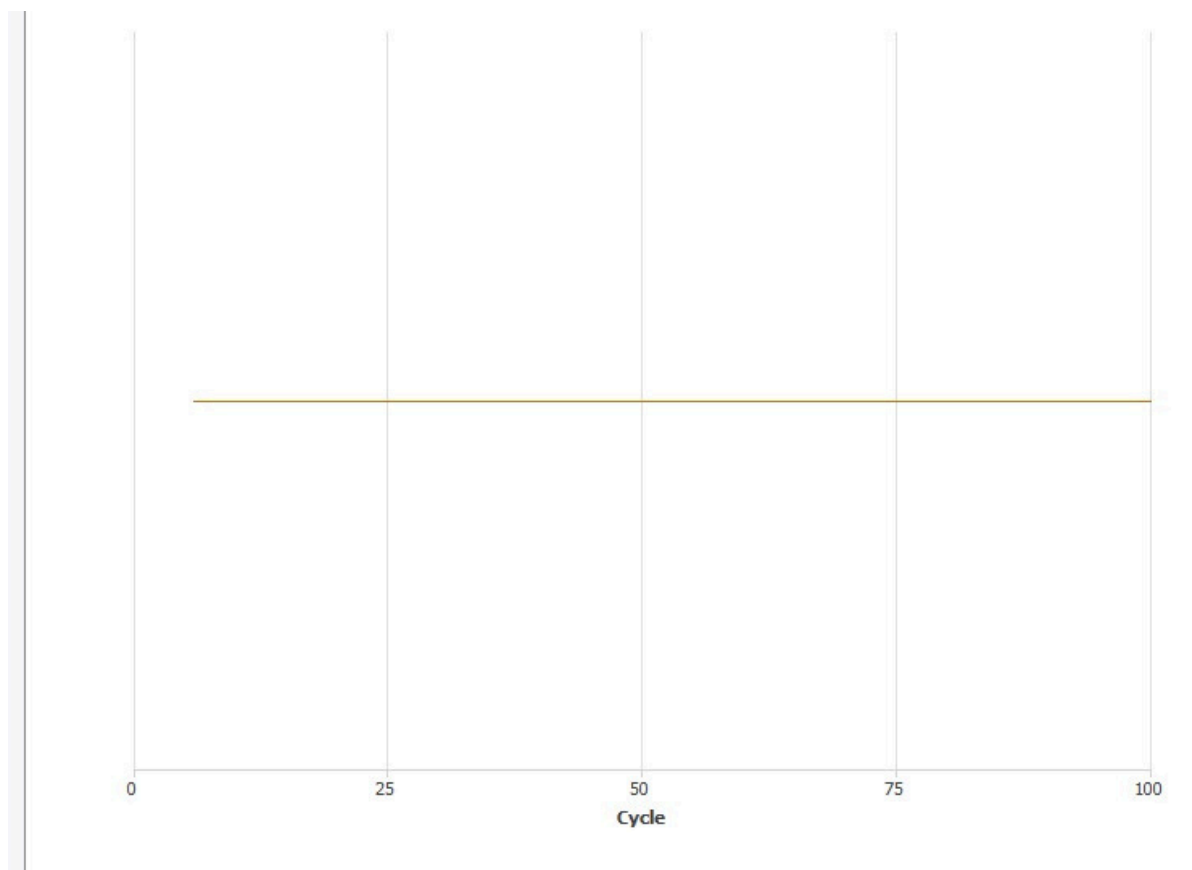


Figure 6.2 : Cache graph for 4-Byte Block Size

- **Graph Observation:** The performance plot stays flat at the 0% baseline. This visually confirms that no Spatial Locality is being exploited, as every access results in a Compulsory Miss.

Case 2: 8-Byte Block Size (2 Words/Line)

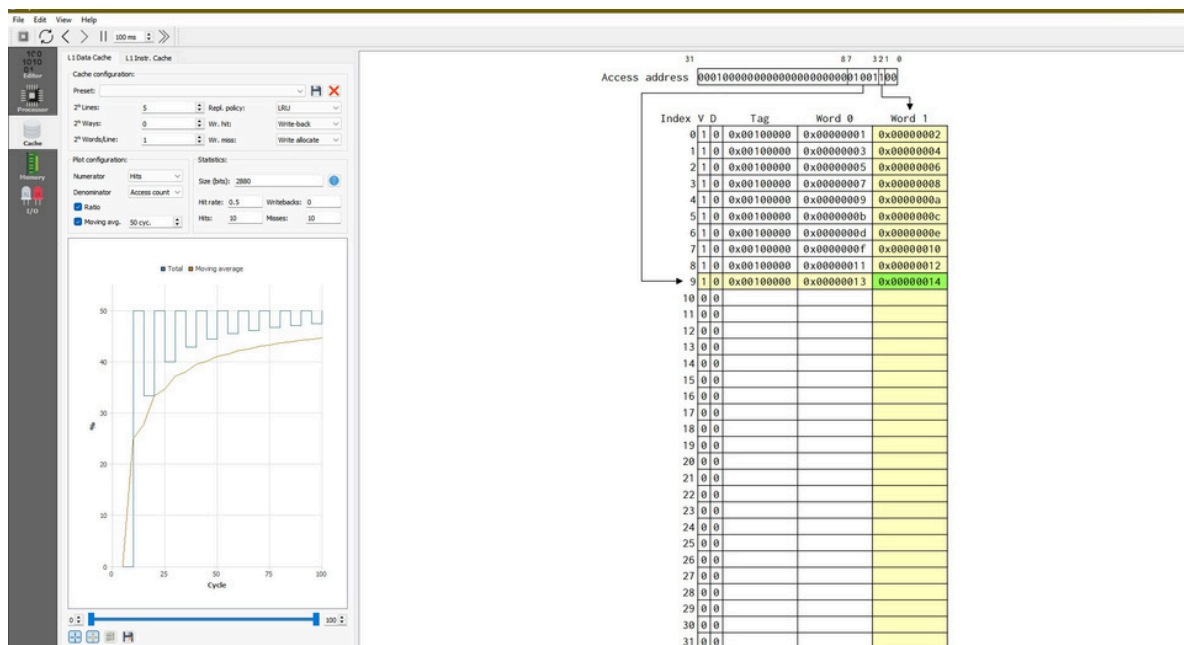


Figure 6.3: Simulation Results for 8-Byte Block Size (Cache Hit)

- **Configuration:** Block size is increased to 8 bytes (2^1 words/line).
- **Results:** Hits: 10, Misses: 10, Hit Rate: 50%.
- **Explanation:** For every miss, the cache now fetches two sequential words. The first word is a miss, but the second word is already in the cache, resulting in a hit.

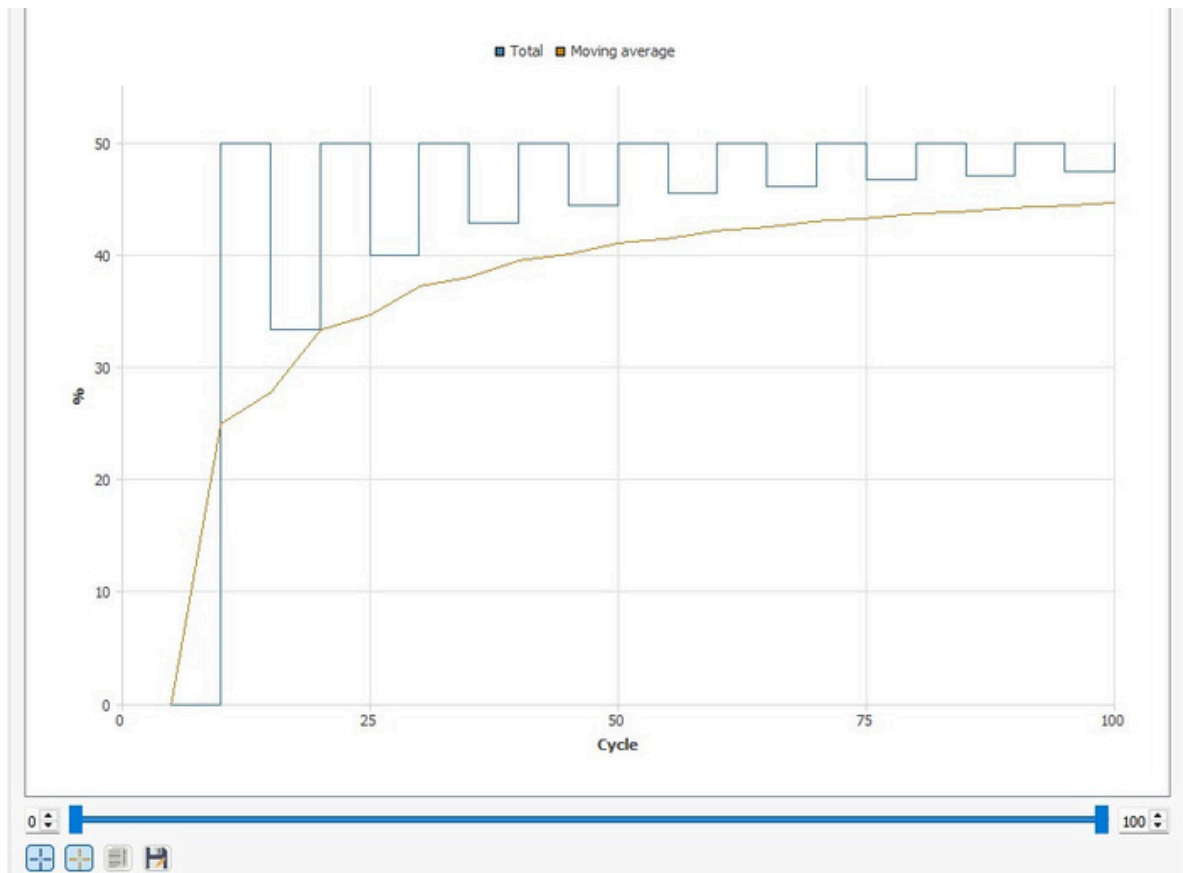


Figure 6.4: Simulation Results for 8-Byte Block Size

- **Graph Observation:** The graph shows a "step" pattern. The moving average begins to climb but remains relatively low, showing that while spatial locality is present, it is not yet fully optimized.

Case 3: 16-Byte Block Size (4 Words/Line)

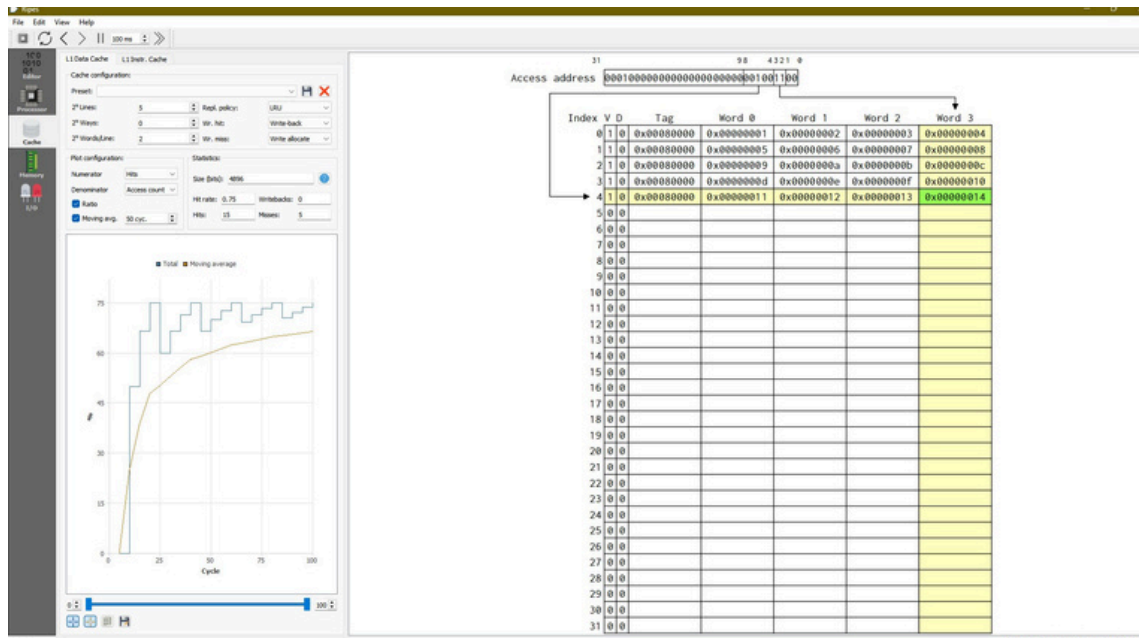


Figure 6.5: Simulation Results for 16-Byte Block Size (Cache Hit)

- **Configuration:** Block size is increased to 16 bytes (2^2 words/line).
- **Results:** Hits: 15, Misses: 5, Hit Rate: 75%.
- **Explanation:** Each miss now pre-fetches three additional words. This significantly reduces the total number of times the processor must wait for the main memory.

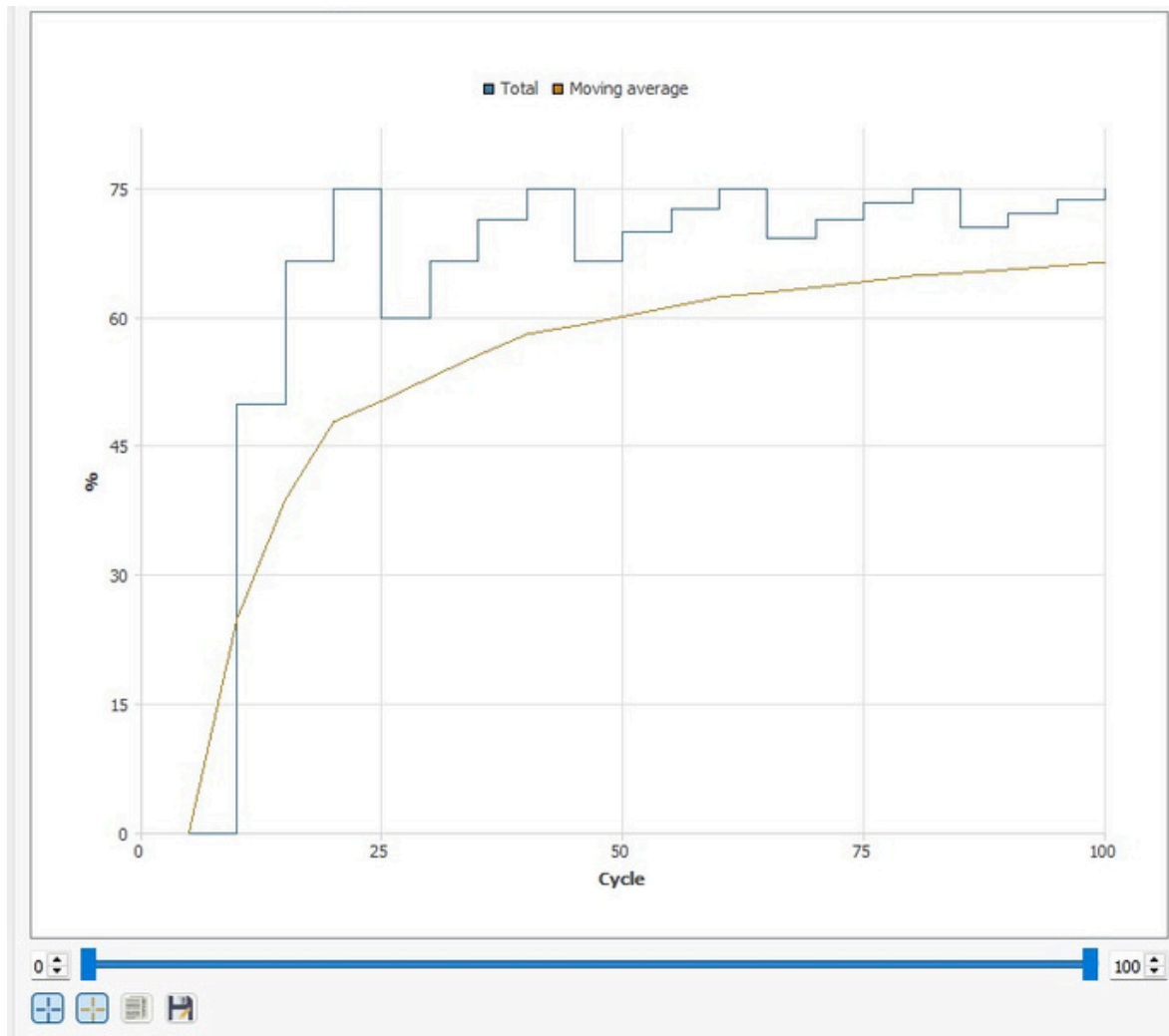


Figure 6.6: Simulation Results for 16-Byte Block Size

- **Graph Observation:** The graph shows a "step" pattern. The moving average begins to climb but remains relatively low, showing that while spatial locality is present, it is not yet fully optimized.

Case 4: 32-Byte Block Size (8 Words/Line)

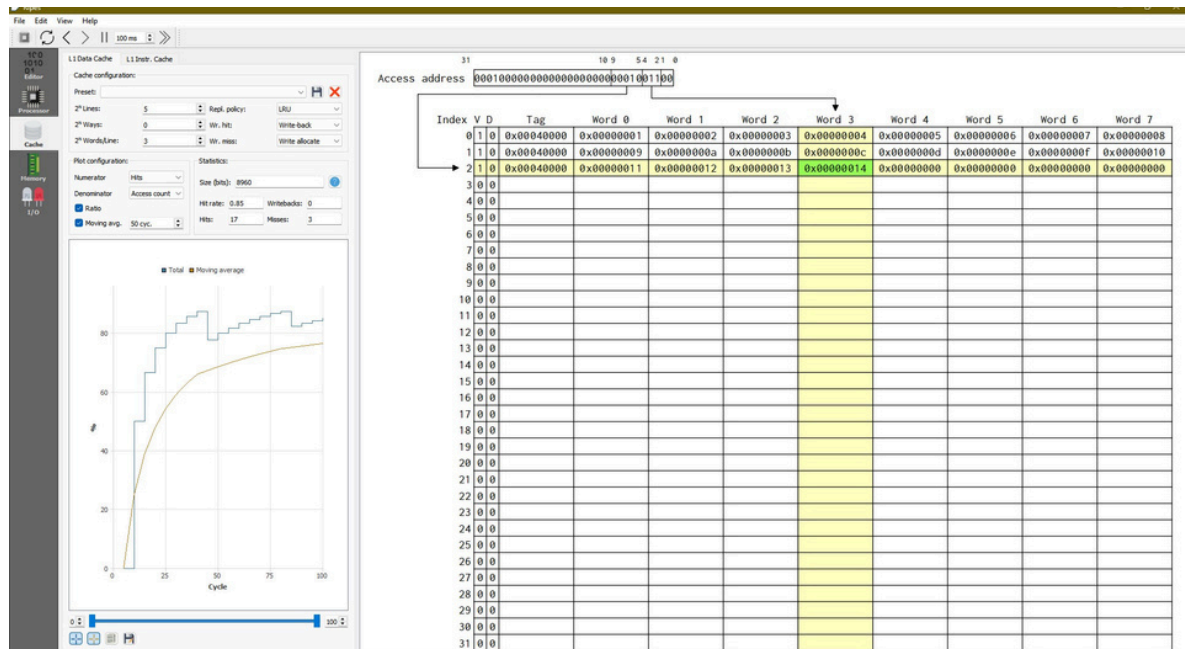


Figure 6.7: Simulation Results for 32-Byte Block Size (Cache Hit)

- **Configuration:** Block size is increased to 32 bytes (2^3 words/line).
- **Results:** Hits: 17, Misses: 3, Hit Rate: 85%.
- **Explanation:** With 8 words per block, 87.5% of the accesses (7 out of 8) in a sequential traversal should ideally be hits. The results closely match this theoretical expectation.

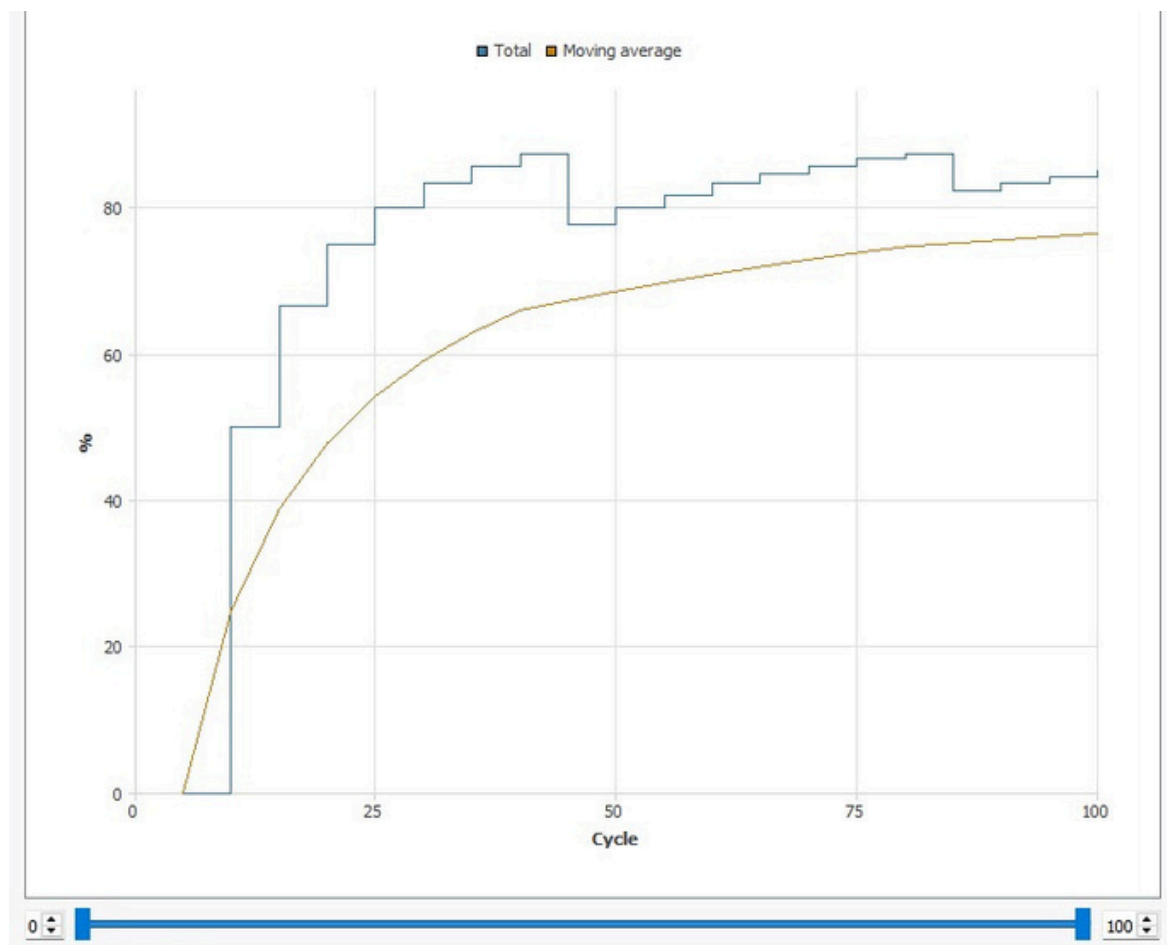


Figure 6.8: Cache graph for 32-Byte Block Size

- **Graph Observation:** The graph begins to plateau at a high level. The "Total" blue line has fewer dips, indicating that the cache misses are becoming rare.

Case 5: 64-Byte Block Size (16 Words/Line)

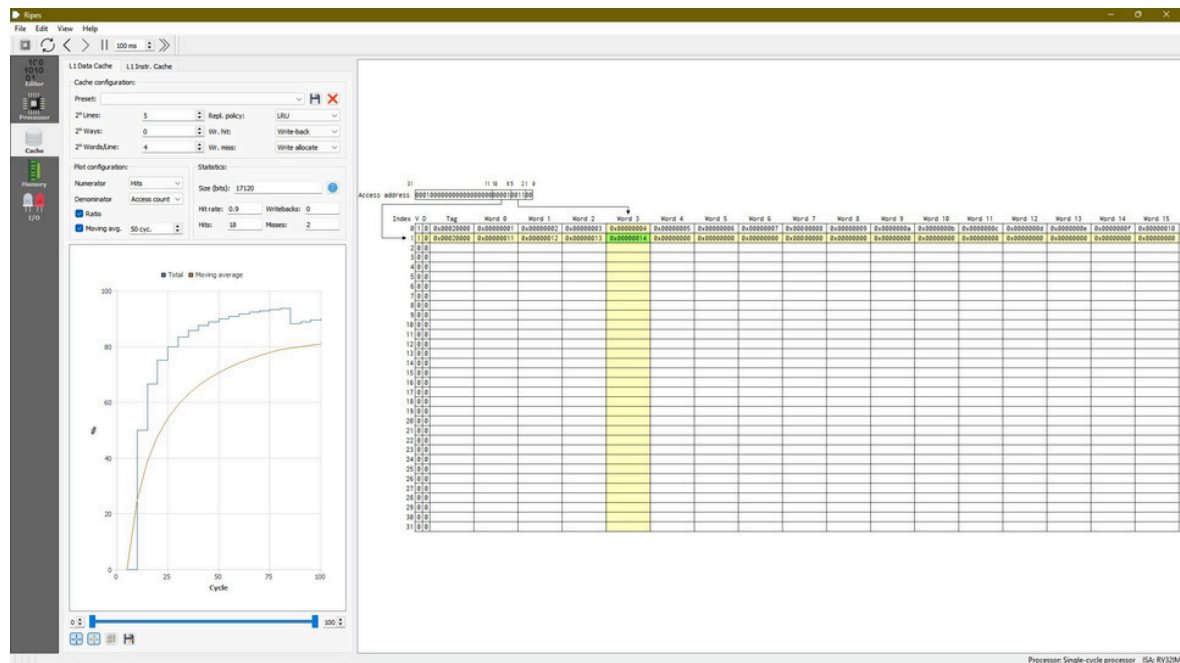


Figure 6.9: Simulation Results for 64-Byte Block Size (Cache Hit)

- **Configuration:** Block size is set to the maximum of 64 bytes (2^4 words/line).
- **Results:** Hits: 18, Misses: 2, Hit Rate: 90%.
- **Explanation:** At this level, a single miss brings 16 words into the cache. For a small array, nearly the entire data set is loaded in just a few misses.

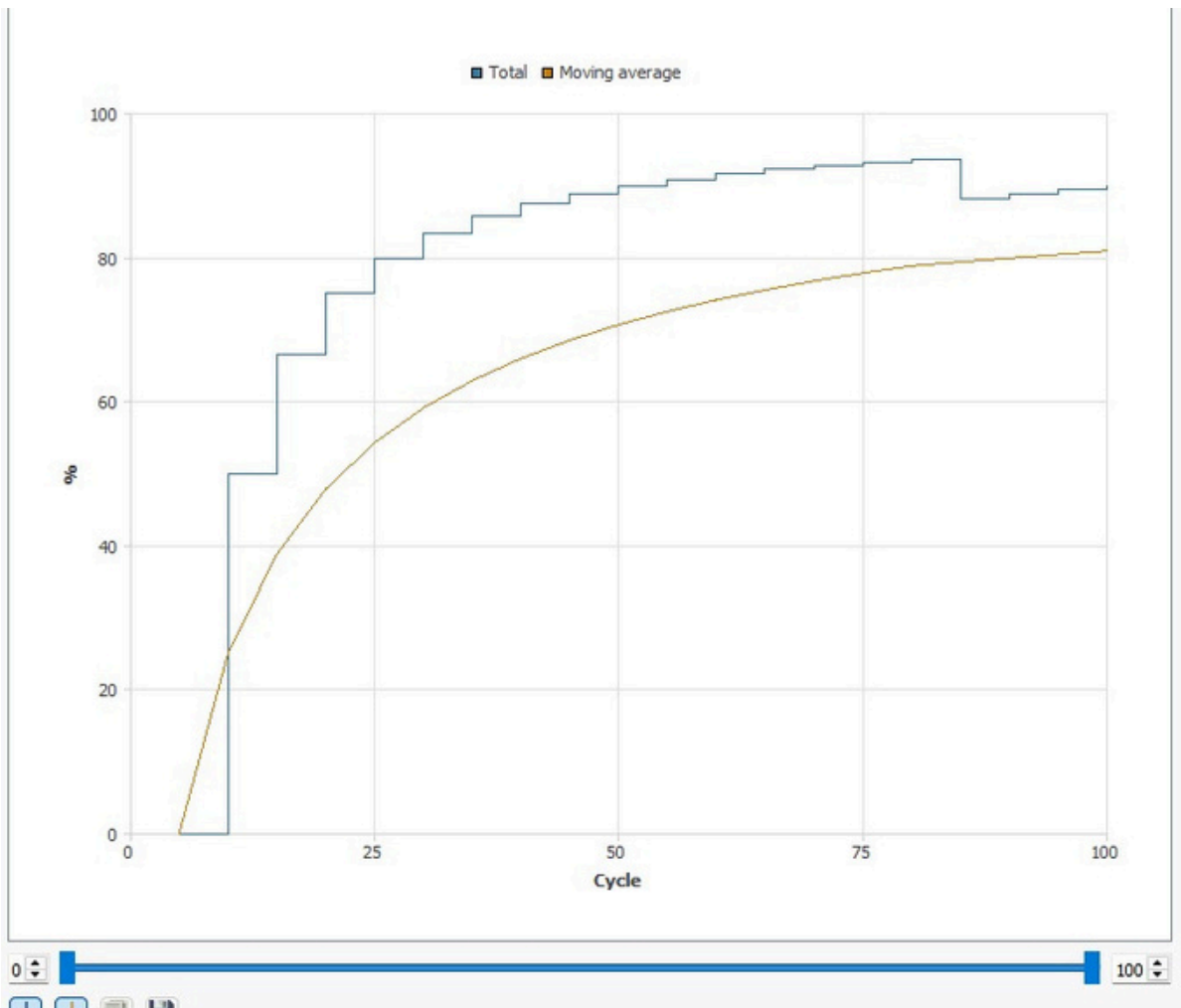


Figure 6.10: Cache graph for 64-Byte Block Size

- **Graph Observation:** The graph shows a very sharp initial climb followed by a steady high-altitude plateau. This confirms that the system has reached the "sweet spot" where the Cache Hit Ratio is maximized.

CHAPTER 7

CONCLUSION

The empirical analysis of cache block size on spatial locality and system hit rates successfully demonstrates the fundamental principles of the memory hierarchy within a processor. Through the use of the Ripes RISC-V ISA Simulator, a complete understanding of how varying the block size impacts the overall Cache Hit Ratio has been achieved. The project not only emphasizes the importance of exploiting spatial locality to reduce compulsory misses but also showcases how an optimized configuration can significantly enhance CPU execution efficiency.

By executing a data-intensive array-traversal algorithm, practical experience was gained in identifying the performance "sweet spot" where system efficiency peaks. The simulation results clearly illustrate the trade-offs between memory bandwidth and cache pollution, providing a strong foundation for learning modern Computer Organization and Architecture (COA). Ultimately, this study stands as a powerful educational tool that reinforces essential concepts of hardware-software interaction and memory management crucial for any aspiring computer engineer.

CHAPTER 8

REFERENCES

1. **Hennessy, J. L., & Patterson, D. A. (2017).** Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann. Provides the fundamental theoretical framework for memory hierarchy and the mathematical analysis of cache design.
2. **Stallings, W. (2022).** Computer Organization and Architecture: Designing for Performance (11th ed.). Pearson. A comprehensive guide defining the principles of spatial locality and the classification of cache misses.
3. **Ripes Visual Simulator. (2026).** GitHub Repository. The official documentation and project website for the visual computer architecture simulator used to model the RISC-V pipeline and cache
4. **Waterman, A., & Asanović, K. (2019).** The RISC-V Reader: An Open Architecture Atlas. Strawberry Canyon. The official reference manual for the RISC-V Instruction Set Architecture (ISA) utilized in the project's assembly source code.
5. **Tanenbaum, A. S., & Austin, T. (2012).** Structured Computer Organization. Pearson. Explains the "Memory Wall" phenomenon and the critical role of cache block size in modern memory management systems.
6. **W3Schools. (2026).** Assembly Language Basics. A general-purpose technical tutorial covering the fundamental logic of assembly programming, branching, and data movement instructions.



Department of Computer Science and Engineering
Vidya Academy of Science and Technology
Thalakkottukara, Thrissur - 680501
(<http://www.vidyaacademy.ac.in>)